

AEGIS

AUTOMATED RISK SHIELD

SOLANA

MVP Development Guide

A complete step-by-step technical blueprint for building the Aegis Solana wallet and token risk scanner from scratch – covering RPC integration, type detection, risk scoring, frontend components, and deployment.

<i>Version</i>	1.0 – MVP
<i>Prepared for</i>	Travy
<i>Stack</i>	Next.js 14 · TypeScript · Tailwind · @solana/web3.js
<i>Target</i>	Working scanner on Vercel

What You're Building

Aegis is a Solana wallet risk scanner. A user pastes any Solana address – a wallet or a token mint (contract address) – and the app does three things:

- **Auto-detects** what type of address it is – wallet vs token mint
- **For wallets:** shows all SPL token holdings, SOL balance, risk flags per token, and an overall risk score
- **For token mints:** shows on-chain metadata – name, symbol, supply, decimals, freeze/mint authority status – and scores the risk

This guide covers every file you need to create to get the MVP running. No fluff – just what you need.

Part 1 – Project Setup

Stack

Library / Tool	Purpose
Next.js 14 (App Router)	Framework – server components + API routes
TypeScript	Type safety across all files
Tailwind CSS	Utility styling
@solana/web3.js	Core RPC connection and account queries
@solana/spl-token	Read token mint and token account data
@metaplex-foundation/mpl-token-metadata	Parse Metaplex metadata PDAs

Initialize the project

```
npx create-next-app@latest aegis-scanner --typescript --tailwind --app
cd aegis-scanner
```

Install Solana dependencies

```
npm install @solana/web3.js @solana/spl-token @metaplex-foundation/mpl-token-metadata
```

Webpack polyfills – add to next.config.js

web3.js uses Node.js modules (crypto, stream, fs) that aren't available in the browser. This tells Next.js to stub them out on the client side:

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  webpack: (config) => {
    config.resolve.fallback = {
      crypto: false,
      stream: false,
      fs: false,
    };
    return config;
  },
};
module.exports = nextConfig;
```

Part 2 – Core Logic: Distinguishing Address Types

This is the most important concept in the whole app. Every Solana address is a base58 public key – 32-44 characters. The address alone doesn't tell you what it is. You have to query the chain using `getAccountInfo()` and read the owner field.

Owner Program → Address Type Mapping

Owner Program ID	Address Type
11111111111111111111111111111111 (System Program)	Regular wallet (EOA)
TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA	SPL Token mint
TokenzQdBNbLqP5VEhdkAS6EPFLC1PHnBqCXEpPxuEb	Token-2022 mint
Anything else	Program / PDA

Detection flow

```
Input: address string
■
■■ isValidPublicKey(address)? → No → Return error
■
▼
Call getAccountInfo(address)
■
■■ Not found → "unknown" (unfunded address)
■
■■ owner == System Program → WALLET path
■
■■ owner == Token Program
■ OR Token-2022 Program → TOKEN MINT path
■
■■ owner == anything else → PROGRAM / UNKNOWN
```

Part 3 – File Structure

```
aegis-scanner/
├── app/
│   ├── page.tsx ← Main UI page
│   ├── layout.tsx ← Root layout
│   ├── api/
│   ├── scan/
│   ├── route.ts ← Core API route (runs server-side)
│   └── components/
│       ├── InputSection.tsx ← Address input + scan button
│       ├── RiskGauge.tsx ← SVG semicircle gauge
│       ├── WalletResults.tsx ← Holdings list + wallet risk flags
│       ├── TokenResults.tsx ← Token mint audit info
│       └── HistoryPanel.tsx ← Sidebar of past scans
├── lib/
│   ├── solana.ts ← All RPC / blockchain logic
│   ├── riskScoring.ts ← Risk score calculation
│   ├── types.ts ← Shared TypeScript interfaces
│   └── .env.local ← RPC URL + API keys (never commit)
```

Part 4 – Types (lib/types.ts)

Define your data contracts first. Everything downstream – the API route, components, and risk scorer – references these interfaces.

```
export type AddressType = "wallet" | "token" | "program" | "unknown";

export interface ScanResult {
  address: string;
  type: AddressType;
  riskScore: number; // 0-100, higher = riskier
  riskLabel: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
  scannedAt: string; // ISO timestamp
  wallet?: WalletData;
  token?: TokenData;
  error?: string;
}

export interface WalletData {
  solBalance: number; // in SOL (not lamports)
  solBalanceUSD: number;
  tokenHoldings: TokenHolding[];
  totalValueUSD: number;
  riskFlags: RiskFlag[];
}

export interface TokenHolding {
  mint: string;
  symbol: string;
  name: string;
  balance: number;
  decimals: number;
  valueUSD: number;
  riskLevel: "SAFE" | "CAUTION" | "DANGER";
  isFrozen: boolean;
  logoUri?: string;
}

export interface TokenData {
  name: string;
  symbol: string;
  decimals: number;
  supply: number; // human-readable
```

```
mintAuthority: string | null; // null = renounced (good)
freezeAuthority: string | null;
isVerified: boolean;
metadataUri?: string;
riskFlags: RiskFlag[];
}

export interface RiskFlag {
  label: string;
  severity: "INFO" | "WARNING" | "DANGER";
  description: string;
}
```

Part 5 – Solana RPC Logic (lib/solana.ts)

This file contains all blockchain interaction. It runs server-side only (called from the API route) so CORS is never an issue.

Address validation + type detection

```
import { Connection, PublicKey, LAMPORTS_PER_SOL } from "@solana/web3.js";
import { TOKEN_PROGRAM_ID, getMint } from "@solana/spl-token";
import { AddressType } from "../types";

const RPC_URL = process.env.SOLANA_RPC_URL ?? "https://api.mainnet-beta.solana.com";
const SYSTEM_PROGRAM = "1111111111111111111111111111111111111111111111111111111111111111";
const TOKEN_PROGRAM = "TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA";
const TOKEN_2022 = "TokenzQdBNbLqP5VEhdkAS6EPFLC1PHnBqCXEpPxuEb";

export const connection = new Connection(RPC_URL, "confirmed");

export function isValidPublicKey(address: string): boolean {
  try { new PublicKey(address); return true; }
  catch { return false; }
}

export async function detectAddressType(address: string): Promise<AddressType> {
  const info = await connection.getAccountInfo(new PublicKey(address));
  if (!info) return "unknown";
  const owner = info.owner.toBase58();
  if (owner === SYSTEM_PROGRAM) return "wallet";
  if (owner === TOKEN_PROGRAM || owner === TOKEN_2022) return "token";
  return "program";
}
```

Wallet data fetching

For wallets, you fetch the SOL balance and all SPL token accounts. The Jupiter strict token list is used to enrich metadata (name, symbol, logo) without needing a paid API key:


```

export async function fetchWalletData(address: string): Promise<WalletData> {
  const pubkey = new PublicKey(address);

  // 1. SOL balance
  const lamports = await connection.getBalance(pubkey);
  const solBalance = lamports / LAMPORTS_PER_SOL;

  // 2. All SPL token accounts
  const tokenAccounts = await connection.getParsedTokenAccountsByOwner(pubkey, {
    programId: TOKEN_PROGRAM_ID,
  });

  // 3. Enrich with Jupiter token list
  let jupiterMap: Record<string, { symbol: string; name: string; logoURI?: string }> = {};
  try {
    const res = await fetch("https://token.jup.ag/strict", {
      next: { revalidate: 3600 }
    });
    const tokens: any[] = await res.json();
    for (const t of tokens) jupiterMap[t.address] = t;
  } catch {}

  // 4. Build holdings
  const holdings = tokenAccounts.value
    .map((acct) => {
      const info = acct.account.data.parsed.info;
      const mint = info.mint;
      const meta = jupiterMap[mint];
      return {
        mint,
        symbol: meta?.symbol ?? mint.slice(0, 6) + "...",
        name: meta?.name ?? "Unknown Token",
        balance: info.tokenAmount.uiAmount ?? 0,
        decimals: info.tokenAmount.decimals,
        valueUSD: 0,
        riskLevel: assessTokenRisk(isFrozen, meta),
        isFrozen: info.state === "frozen",
        logoUri: meta?.logoURI,
      };
    });
}

```

```

};
}))
.filter((h) => h.balance > 0);
return {
  solBalance,
  solBalanceUSD: 0, // plug in Jupiter price API – see Part 8
  tokenHoldings: holdings,
  totalValueUSD: 0,
  riskFlags: buildWalletRiskFlags(holdings),
};
}

```

Token mint data fetching

```

export async function fetchTokenData(address: string): Promise<TokenData> {
  const pubkey = new PublicKey(address);
  const mint = await getMint(connection, pubkey);
  const mintAuthority = mint.mintAuthority?.toBase58() ?? null;
  const freezeAuthority = mint.freezeAuthority?.toBase58() ?? null;
  const supply = Number(mint.supply) / Math.pow(10, mint.decimals);
  // Try Jupiter single-token endpoint for metadata
  let name = "Unknown Token", symbol = "???", isVerified = false;
  let metadataUri: string | undefined;

  try {
    const r = await fetch(`https://tokens.jup.ag/token/${address}`);
    if (r.ok) {
      const d = await r.json();
      name = d.name; symbol = d.symbol;
      metadataUri = d.logoURI; isVerified = true;
    }
  } catch {}

  return {
    name, symbol,
    decimals: mint.decimals,
    supply,
    mintAuthority,
    freezeAuthority,
    isVerified,
    metadataUri,
    riskFlags: buildTokenRiskFlags({ mintAuthority, freezeAuthority, supply, isVerified }),
  };
}

```

Part 6 – Risk Scoring (lib/riskScoring.ts)

Scores are deterministic – based on actual on-chain signals, not random. Each DANGER flag adds 30 points, each WARNING adds 15, each renounced-authority INFO flag subtracts 10. Score is clamped 0–100.

```
import { RiskFlag } from "../types";

export function calculateRiskScore(flags: RiskFlag[]): number {
  let score = 0;
  for (const flag of flags) {
    if (flag.severity === "DANGER") score += 30;
    if (flag.severity === "WARNING") score += 15;
    if (flag.severity === "INFO" && flag.label.includes("Renounced")) score -= 10;
  }
  return Math.max(0, Math.min(100, score));
}

export function getRiskLabel(score: number): "LOW"|"MEDIUM"|"HIGH"|"CRITICAL" {
  if (score <= 20) return "LOW";
  if (score <= 50) return "MEDIUM";
  if (score <= 75) return "HIGH";
  return "CRITICAL";
}
```

Risk flag signals – what triggers what

Signal	Type	Severity	Points
Mint authority still active	Token	DANGER	+30
Freeze authority still active	Token	DANGER	+30
Frozen token account in wallet	Wallet	DANGER	+30
Token not in Jupiter verified list	Both	WARNING	+15
Supply > 1 trillion	Token	WARNING	+15
5+ unknown tokens in wallet	Wallet	WARNING	+15
Mint authority renounced	Token	INFO	-10
Freeze authority renounced	Token	INFO	-10

Part 7 – API Route (app/api/scan/route.ts)

This is the Next.js server-side handler. It runs on the server, so your RPC key stays private and CORS is never a problem. The frontend calls POST /api/scan.

```
import { NextRequest, NextResponse } from "next/server";
import { isValidPublicKey, detectAddressType,
fetchWalletData, fetchTokenData } from "@lib/solana";
import { calculateRiskScore, getRiskLabel } from "@lib/riskScoring";
import { ScanResult } from "@lib/types";

export async function POST(req: NextRequest) {
  try {
    const { address } = await req.json();
    if (!address || !isValidPublicKey(address.trim())) {
      return NextResponse.json({ error: "Invalid Solana address" }, { status: 400 });
    }

    const trimmed = address.trim();
    const addressType = await detectAddressType(trimmed);

    if (addressType === "unknown" || addressType === "program") {
      return NextResponse.json({
        address: trimmed, type: addressType,
        riskScore: 0, riskLabel: "LOW",
        scannedAt: new Date().toISOString(),
        error: addressType === "unknown"
          ? "Address has no on-chain data – may never have been funded."
          : "This is a program/smart contract, not a wallet or token mint.",
      } as ScanResult);
    }

    let result: ScanResult = {
      address: trimmed, type: addressType,
      riskScore: 0, riskLabel: "LOW",
      scannedAt: new Date().toISOString(),
    };

    if (addressType === "wallet") {
      const walletData = await fetchWalletData(trimmed);
      result.wallet = walletData;
      result.riskScore = calculateRiskScore(walletData.riskFlags);
      result.riskLabel = getRiskLabel(result.riskScore);
    }
  }
}
```

```
}  
if (addressType === "token") {  
  const tokenData = await fetchTokenData(trimmed);  
  result.token = tokenData;  
  result.riskScore = calculateRiskScore(tokenData.riskFlags);  
  result.riskLabel = getRiskLabel(result.riskScore);  
}  
return NextResponse.json(result);  
} catch (err: any) {  
  if (err.message?.includes("429")) {  
    return NextResponse.json(  
      { error: "RPC rate limit hit. Retry in a few seconds." }, { status: 429 }  
    );  
  }  
  return NextResponse.json({ error: "Scan failed." }, { status: 500 });  
}  
}
```


Part 9 – RPC Providers & Rate Limits

The public mainnet RPC is heavily rate-limited. For development it's fine, but for production you need a private endpoint. All options below have a free tier.

Provider	Free Tier	Best Feature
Helius	100k req/day	DAS API for rich token metadata – highly recommended
QuickNode	10M credits/month	Reliable, good observability
Alchemy	Free tier	Good for webhooks and event-driven features
Triton	Free plan	Fast, Solana-native infrastructure

Environment variables (.env.local)

```
SOLANA_RPC_URL=https://mainnet.helius-rpc.com/?api-key=YOUR_KEY
HELIUS_API_KEY=YOUR_KEY
```

Helius DAS API – richer token metadata (recommended)

Instead of parsing raw Metaplex PDAs manually, Helius's `getAsset` endpoint returns name, symbol, image, description, and supply in a single call:

```
const res = await fetch(
  `https://mainnet.helius-rpc.com/?api-key=${process.env.HELIUS_API_KEY}`,
  {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      jsonrpc: "2.0", id: "1",
      method: "getAsset",
      params: { id: mintAddress },
    }),
  }
);
const { result } = await res.json();
// result.content.metadata.name
// result.content.metadata.symbol
// result.content.links.image
```

Part 10 – Frontend Components

RiskGauge.tsx – SVG semicircle gauge

Uses a half-circle SVG path with strokeDashoffset to animate the score. Color changes with riskLabel.

```
interface Props { score: number; label: string; }

export default function RiskGauge({ score, label }: Props) {
  const radius = 80, cx = 100, cy = 100;
  const circumference = Math.PI * radius;
  const offset = circumference - (score / 100) * circumference;
  const colors: Record<string, string> = {
    LOW: "#10b981", MEDIUM: "#f59e0b",
    HIGH: "#ef4444", CRITICAL: "#dc2626",
  };
  const color = colors[label] ?? "#6b7280";
  return (
    <div style={{ display: "flex", flexDirection: "column", alignItems: "center" }}>
      <svg width="200" height="120" viewBox="0 0 200 120">
        {/* Track */}
        <path d="M 20,100 A 80,80 0 0,1 180,100"
          fill="none" stroke="#1e1e2e" strokeWidth="16" strokeLinecap="round" />
        {/* Fill */}
        <path d="M 20,100 A 80,80 0 0,1 180,100"
          fill="none" stroke={color} strokeWidth="16" strokeLinecap="round"
          strokeDasharray={circumference} strokeDashoffset={offset}
          style={{ transition: "stroke-dashoffset 0.8s ease" }} />
        <text x="100" y="90" textAnchor="middle" fill="white" fontSize="28" fontWeight="bold">
          {score}
        </text>
        <text x="100" y="108" textAnchor="middle" fill="#6b7280" fontSize="11">/ 100</text>
      </svg>
    </div>
  );
}
```

page.tsx – Main scan flow

The page manages scan state, calls POST /api/scan, and renders the correct result component based on address type:


```
"use client";
import { useState } from "react";
import { ScanResult } from "@lib/types";
export default function Home() {
  const [result, setResult] = useState<ScanResult | null>(null);
  const [loading, setLoading] = useState(false);
  const scan = async (address: string) => {
    setLoading(true);
    const res = await fetch("/api/scan", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ address }),
    });
    const data = await res.json();
    setResult(data);
    setLoading(false);
  };
  return (
    <main>
      <InputSection onScan={scan} isLoading={loading} />
      {result?.wallet && <WalletResults data={result.wallet} address={result.address} />}
      {result?.token && <TokenResults data={result.token} address={result.address} />}
    </main>
  );
}
```

Part 11 – Deployment (Vercel)

Aegis is a Next.js app. Vercel is the fastest deploy path – the API route runs as a serverless function so your RPC key is never exposed to the browser.

```
npm install -g vercel  
vercel
```

Add SOLANA_RPC_URL and HELIUS_API_KEY in Vercel dashboard → Project Settings → Environment Variables.

Part 12 – MVP Checklist

Use this to track progress. The MVP is done when every box is checked.

Project & Config

- Next.js initialized with TypeScript + Tailwind
- Webpack polyfills added to next.config.js
- .env.local created with SOLANA_RPC_URL

Core Logic

- lib/types.ts – all interfaces defined
- lib/solana.ts – isValidPublicKey working
- lib/solana.ts – detectAddressType working (returns wallet/token/unknown/program)
- lib/solana.ts – fetchWalletData returns SOL balance + token holdings
- lib/solana.ts – fetchTokenData returns supply, mint/freeze authority, metadata
- lib/riskScoring.ts – calculateRiskScore and getRiskLabel working

API Layer

- app/api/scan/route.ts – POST handler validates input
- POST handler detects type and routes to correct fetch function
- POST handler returns correct ScanResult shape
- Error handling for RPC failures and rate limits

Frontend

- InputSection.tsx – address input + scan button renders
- RiskGauge.tsx – SVG gauge shows correct score and color
- WalletResults.tsx – SOL balance, token list, risk flags visible
- TokenResults.tsx – name, symbol, supply, authorities, risk flags visible
- Scan history stored in state, shown in sidebar

Testing

- Tested with a known wallet address
- Tested with USDC mint: EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v

- Tested with an invalid address – shows correct error
- Wallet with DANGER tokens shows high risk score
- Token with active mint authority shows DANGER flag

Deploy

- Vercel project created
- Env vars set in Vercel dashboard
- Production scan tested on live URL

Part 13 – Post-MVP Roadmap

Limitation in MVP	Fix for v2+
No USD token pricing	Integrate Jupiter Price API v6 into <code>fetchWalletData</code>
Token metadata can be missing	Replace Jupiter fallback with Helius <code>getAsset</code>
No transaction history	Add <code>getSignaturesForAddress</code> + parse instruction types
No mixer/drainer detection	Build against known drainer address blocklist
No holder count	Fetch from Helius <code>getTokenLargestAccounts</code>
History lost on reload	Persist to <code>localStorage</code> or Supabase
No Token-2022 extension parsing	Parse extensions from raw account data
No wallet labeling	Integrate Solana FM or Step.Finance label APIs